

L7: FUNDAMENTOS DE GIT Y GITHUB - ANÁLISIS DE CASO

Proyecto: Wallet Digital | Estudiante: Javier Rodriguez

1. INFORME DE ANÁLISIS

1.1 Importancia del Control de Versiones

Git es un sistema de control de versiones distribuido fundamental para:

- **Historial completo:** Cada cambio en el código queda registrado con autor, fecha y descripción
- **Colaboración en equipo:** Múltiples desarrolladores trabajan sin pisar cambios ajenos
- **Recuperación de errores:** Volver a versiones anteriores en segundos
- **Trazabilidad:** Saber quién cambió qué, cuándo y por qué

En nuestro proyecto **wallet digital**, Git permite que si el módulo de pagos falla, revertimos en minutos sin perder todo el trabajo.

1.2 Beneficios de GitHub para Equipos Distribuidos

GitHub añade a Git:

- **Repositorio remoto centralizado:** El código está en la nube, accesible desde cualquier lugar
- **Pull Requests:** Revisión de código antes de mezclar cambios (calidad)
- **Issues y Proyectos:** Gestión de tareas y seguimiento de bugs
- **Automatización CI/CD:** Pruebas automáticas en cada push
- **Documentación:** README, wikis y páginas del proyecto

Ejemplo práctico: En DevTech Solutions, cuando Juan desde Buenos Aires hace un PR con mejoras al módulo de seguridad, María desde Santiago revisa, comenta y aprueba. Sin GitHub, habrían perdido horas sincronizando.

1.3 Limitaciones de GitHub y Mitigación

Limitación	Solución
Curva de aprendizaje	Documentación, capacitación en equipo

Conflictos al mergear	Ramas por feature y comunicación constante
Seguridad de datos	Branch protection, contraseñas fuertes, 2FA
Límite de almacenamiento	Git LFS para archivos grandes

2. ESTRATEGIA DE RAMAS Y FLUJO DE TRABAJO

2.1 Modelo GitFlow - Implementación en Wallet Digital



Ramas principales:

- **main**: Solo código en producción (versiones estables)
- **develop**: Integración de nuevas features

Ramas secundarias:

- **feature/xxx**: Nuevas funcionalidades (duran días)
- **bugfix/xxx**: Correcciones rápidas

2.2 Protección de Ramas

En nuestro repositorio configuramos:

main:

- ✓ Requiere Pull Request
- ✓ Mínimo 1 revisor
- ✓ Checks automáticos pasen
- ✗ No push directo

develop:

- ✓ Requiere Pull Request
- ✓ Mínimo 1 revisor

2.3 Flujo Práctico (Paso a Paso)

Desarrollador quiere agregar 2FA:

bash

1. Desde develop, crear feature

git checkout develop

git pull origin develop

git checkout -b feature/2fa

2. Hacer cambios

... editar archivos ...

3. Commit y push

git add .

git commit -m "feat: Agregar autenticación de dos factores"

git push origin feature/2fa

4. En GitHub: Crear Pull Request

5. Revisor aprueba

6. Merge automático

git checkout develop

git pull origin develop

7. Cuando está listo para producción:

git checkout main

git merge develop

git tag -a v1.0.1 -m "Release: Agregada 2FA"

git push origin main --tags

3. PLAN DE OPTIMIZACIÓN

3.1 Archivo .gitignore

Dependencias

node_modules/

npm-debug.log

yarn.lock

Archivos de configuración sensibles

.env

.env.local

.env*.local

Archivos del sistema

.DS_Store

Thumbs.db

*.swp

```
# Logs
logs/
*.log
```

```
# Builds
dist/
build/
```

En wallet digital - No queremos que suban:

- Contraseñas de API (`.env`)
- Archivos temporales
- Dependencias (se instalan con `npm install`)

3.2 Git LFS para Archivos Grandes

Si tuviéramos videos o datos:

```
bash
git lfs install
git lfs track "*.mp4"
git add .gitattributes
git add video.mp4
git commit -m "Add demo video with LFS"
git push origin main
```

3.3 Optimización de Commits

ANTES (malo):

```
aaa
cambios
fix
fix2
fix3
listo
```

DESPUÉS (bueno):

```
feat: Implementar módulo de transacciones
refactor: Mejorar validación de saldo
fix: Corregir error en cálculo de intereses
docs: Actualizar README con instrucciones de setup
```

3.4 Rebase Interactivo

Limpiar 5 commits en uno:

```
bash
```

```
git rebase -i HEAD~5
```

```
# Cambiar "pick" por "squash" en los últimos 4
```

```
# Guardar y listo: 1 commit limpio
```

3.5 Buenas Prácticas de Performance

✓ Commits pequeños y atómicos (1 feature = 1 commit)

✓ Mensajes claros (**feat:**, **fix:**, **docs:**)

✓ Deletear ramas después de mergear

✓ Rebase antes de PR para historial lineal

✓ CI/CD para detectar issues temprano

4. DOCUMENTACIÓN DE SEGURIDAD

4.1 Riesgos en Repositorio Abierto

Riesgos identificados:

1. **Exposición de secretos:** Contraseñas, API keys en el código
2. **Acceso no autorizado:** Cualquiera puede ver/clonar (si es público)
3. **Inyección maliciosa:** PRs con código malicioso
4. **Cambios no rastreados:** Commits sin verificar
5. **Dependencias vulnerables:** Librerías con bugs de seguridad

4.2 Estrategias de Mitigación

Para DevTech Solutions recomendamos:

A. Secretos y Variables

```
bash
```

```
# ✓ CORRECTO: Usar .env
```

```
API_KEY=${process.env.API_KEY}
```

```
# ✗ INCORRECTO: Hardcodear
```

```
API_KEY="sk-1234567890abcdef"
```

B. Autenticación en GitHub

```
bash
```

```
# Usar tokens en lugar de contraseña
```

```
git remote set-url origin https://[TOKEN]@github.com/tuzkahr-alt/wallet-digital.git
```

C. Protección de Ramas

-  Requiere reviews antes de mergear
-  Protege main de cambios directos
-  Requiere que checks pasen

D. Verificación de Commits

bash

Firmar commits con GPG

git commit -S -m "feat: agregar transacciones"

git log --show-signature

E. Análisis de Dependencias

bash

Detectar vulnerabilidades

npm audit

npm audit fix

4.3 Política de Seguridad para Wallet Digital

POLÍTICA DE SEGURIDAD - Wallet Digital

1. CREDENCIALES

- Nunca commitear .env o contraseñas
- Usar GitHub Secrets para CI/CD
- Rotar contraseñas mensualmente

2. ACCESO

- Solo 2 maintainers pueden mergear a main
- Todos los PRs requieren 1 revisor
- 2FA obligatorio para todos

3. MONITOREO

- Revisar dependencias semanalmente
- Activar alerts de GitHub Security
- Auditar commits mensuales

4. RESPUESTA A INCIDENTES

- Si se filtra secreto: rotar inmediatamente
- Si hay código malicioso: revertir + investigar
- Notificar en 24 horas

5. EVALUACIÓN DE LA EXPERIENCIA DEL USUARIO EN GITHUB

5.1 Análisis de la Interfaz

Fortalezas:

-  Panel de control intuitivo (dashboard)
-  Búsqueda potente de repositorios
-  Visualización clara de PRs e Issues
-  Gráficos de contribuciones

Debilidades:

-  Menú de configuración disperso
-  Difícil encontrar Secrets guardados
-  Notificaciones a veces confusas

5.2 Mejoras en Documentación

Para wallet digital, recomendamos agregar:

README.md mejorado:

markdown

 **Wallet Digital**

Billetera digital segura para transacciones.

 **Requisitos**

- Node.js 16+

- npm 8+

 **Inicio Rápido**

git clone <https://github.com/tuzkahr-alt/wallet-digital.git>

cd wallet-digital

npm install

npm run dev

 **Documentación**

- [Setup Inicial](./docs/setup.md)

- [API Reference](./docs/api.md)

- [Contributing](./CONTRIBUTING.md)

 **Seguridad**

Ver [\[SECURITY.md\]\(./SECURITY.md\)](#)

 **Soporte**

Issues: github.com/tuzkahr-alt/wallet-digital/issues

5.3 Optimizaciones de Usabilidad

Mejora	Implementación
Templates de Issue	Crear <code>.github/ISSUE_TEMPLATE/bug.md</code>
Templates de PR	Crear <code>.github/PULL_REQUEST_TEMPLATE.md</code>
Etiquetas	Usar <code>bug</code> , <code>feature</code> , <code>documentation</code>
Milestones	Agrupar issues por release (v1.0, v1.1)
Projects	Tablero Kanban para tareas

6. ENLACE Y INSTRUCCIONES DEL REPOSITORIO

REPOSITORIO GITHUB

URL: <https://github.com/tuzkahr-alt/wallet-digital>

Cómo Clonar y Usar

bash

Clonar el repositorio

`git clone https://github.com/tuzkahr-alt/wallet-digital.git`

Entrar en la carpeta

`cd wallet-digital`

Instalar dependencias

`npm install`

Crear rama personal para desarrollar

`git checkout -b feature/tu-feature`

Hacer cambios y commitear

`git add .`

`git commit -m "feat: descripción"`

Subir cambios

`git push origin feature/tu-feature`

En GitHub: crear Pull Request

Estado Actual del Repositorio

-  **main**: Código estable en producción
-  **develop**: Integración de nuevas features
-  **.gitignore**: Configurado (node_modules, .env, logs)
-  **Commits**: Historial limpio y descriptivo
-  **Ramas**: feature/2fa, bugfix/validacion

Estructura del Repositorio

```
wallet-digital/  
├── index.html      # Página principal  
├── CSS/            # Estilos  
├── JS/             # Lógica JavaScript  
├── .gitignore      # Archivos ignorados  
├── README.md       # Documentación  
└── .git/           # Historial de Git
```

Próximos Pasos

1. Proteger rama main (requiere reviews)
2. Configurar GitHub Actions para CI/CD
3. Agregar tests automáticos
4. Documentar API completamente
5. Setup de Secrets para variables sensibles

CONCLUSIÓN

La implementación de Git y GitHub en **wallet digital** permite:

-  Control total de versiones
-  Colaboración segura en equipo
-  Trazabilidad completa de cambios
-  Automatización de procesos
-  Código más seguro y confiable

Documento generado: Junio 2024

Estudiante: Javier Rodriguez

Repositorio: <https://github.com/tuzkahr-alt/wallet-digital>